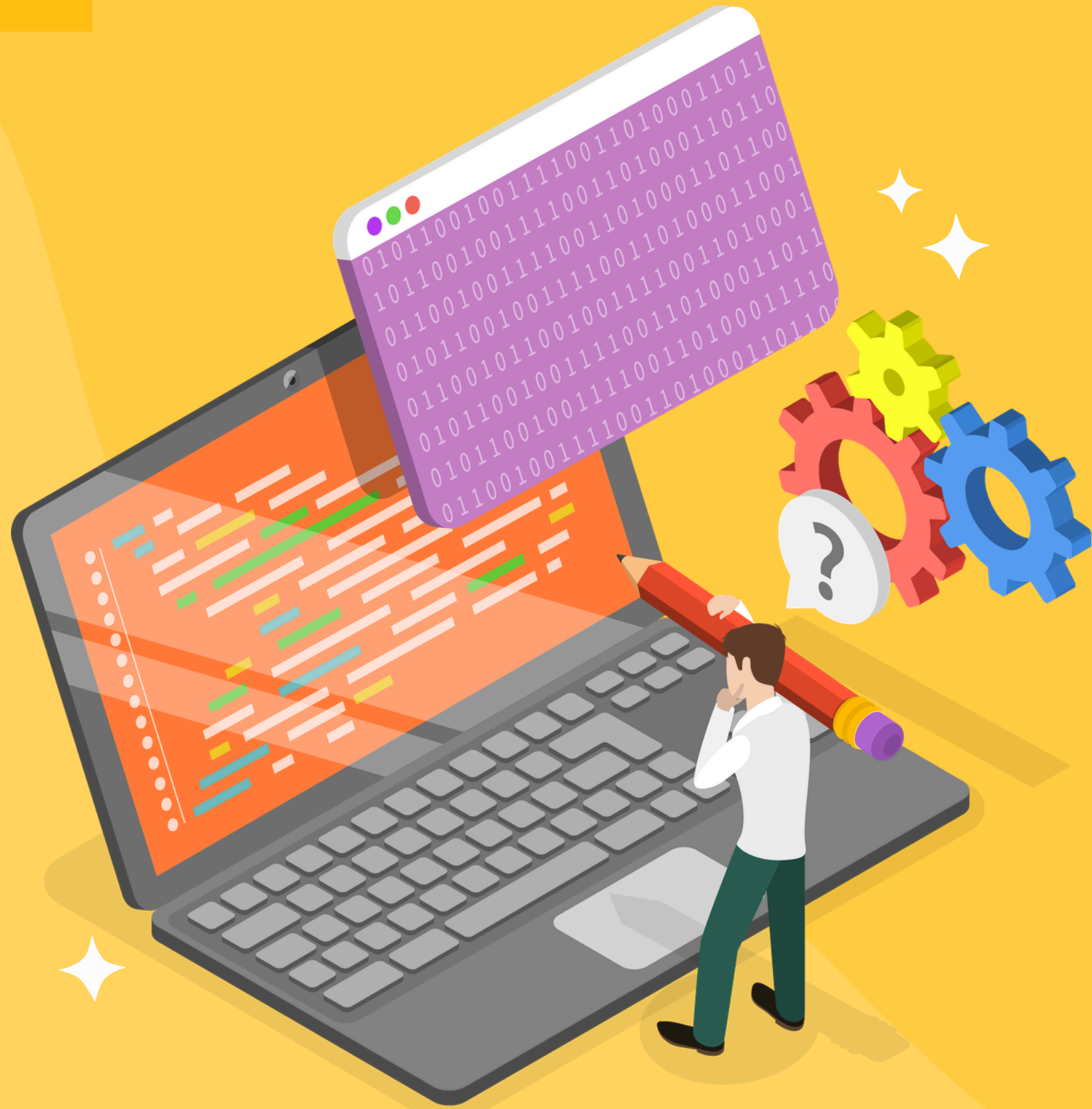


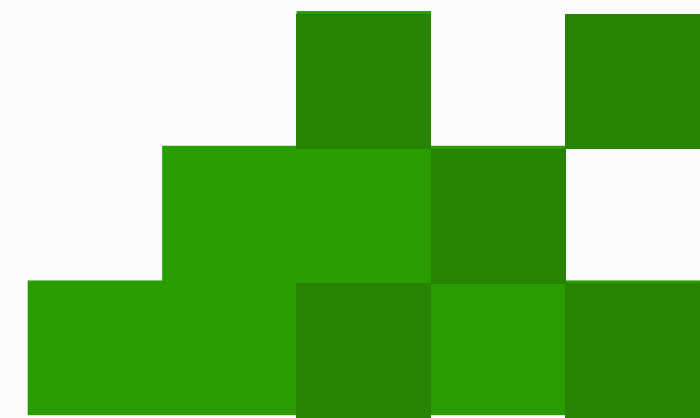


Modern Web Architecture

Non-Blocking Web Programming

and Functional Programming
Paradigm

Presented By: **BYTECLUB**





JavaScript runs on one thread.



BLOCKING

Request A (slow) B waits C waits D waits

NON-BLOCKING (event loop)

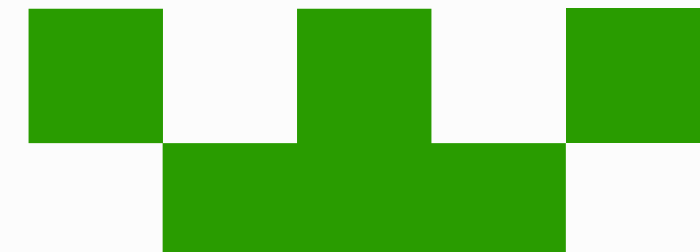
A start B start C start D start

callbacks resolve as I/O completes, in any order

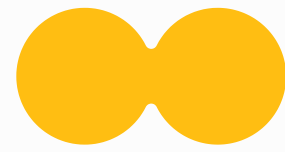
Same single thread – the difference is who waits, and for what.
Non-blocking ≠ multi-threaded. It means: never idle-wait on I/O.

Why this matters

- One call stack, no parallel execution
- A slow task can freeze the whole app
- Node.js delegates I/O instead of waiting
- Functional style keeps that flow predictable



What is Functional Programming?



Functional Programming Languages



Clojure



Common Lisp



Erlang



Haskell



Scala

> hackr.io



Functions as values

Passed, returned, stored — like any other data



No side effects

Same input always produces the same output



Immutable data

State is never changed, only replaced



Composition

Small functions combine into pipelines

Immutability

- Data is never changed in place
- Updates create a new copy instead
- Prevents subtle shared-state bugs

`cart (v1)`
`[pizza, soda]`

spread copy

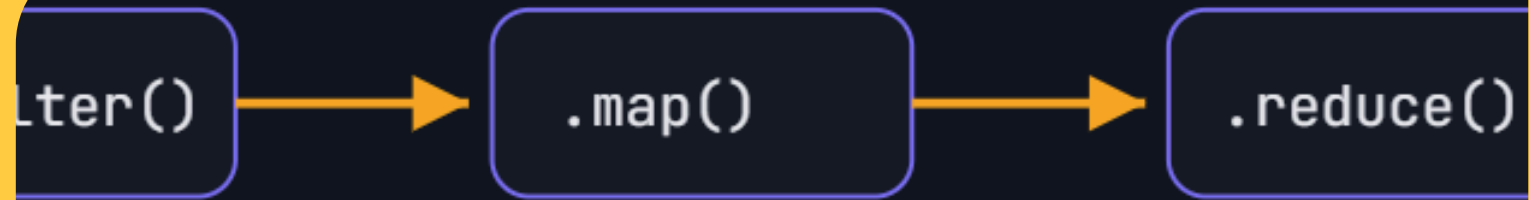
`cart (v2) – new`
`[pizza, soda, salad]`

`cart (v1)` still exists, unchanged
→ old references stay valid: safe for concurrent reads



```
// mutating (avoid)
cart.push(newItem);

// immutable update
const nextCart = [...cart, newItem];
```



```
const total = orders
  .filter((o) => o.paid)
  .map((o) => o.amount)
  .reduce((sum, amount) => sum + amount, 0);
```

Higher-Order Functions & Composition

A function that takes or returns another function. Chaining small pure functions builds a pipeline.





Declarative Approach

vs. Imperative

Imperative

```
let arr = [1, 2, 3, 4, 5],  
arr2 = [];  
  
for (var i=0; i<arr.length; i++) {  
  arr2[i] = arr[i]*2;  
}  
  
console.log(arr2);
```

Declarative

```
let arr = [1, 2, 3, 4, 5];  
  
arr2 = arr.map(function(v, i) {  
  return v * 2;  
});  
  
console.log(arr2);
```

Declarative Over Imperative? For Beginners [4]



Say what you want, not how to do it



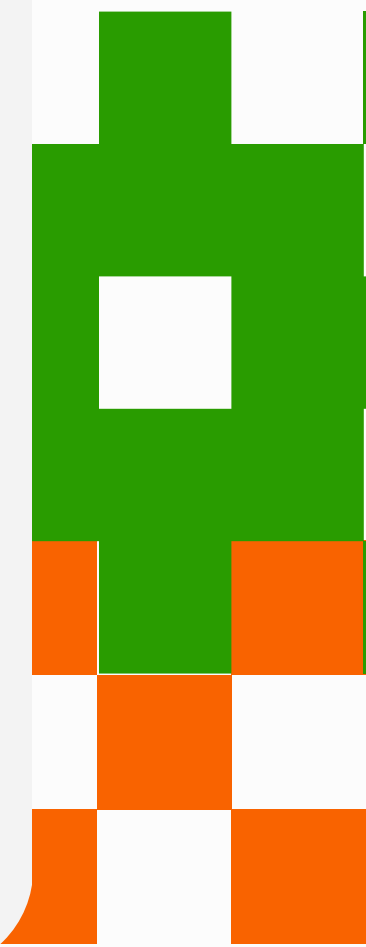
No for loops, no mutable counters



map(), filter(), reduce()



Result: clean, self-documenting code





Function Composition

- Combine small, pure functions into a pipeline
- Avoid giant, monolithic functions
- Chain inputs → outputs, testable at every step
- Key for processing streaming data



01

Function composition is associative

Function composition is closed under composition

02

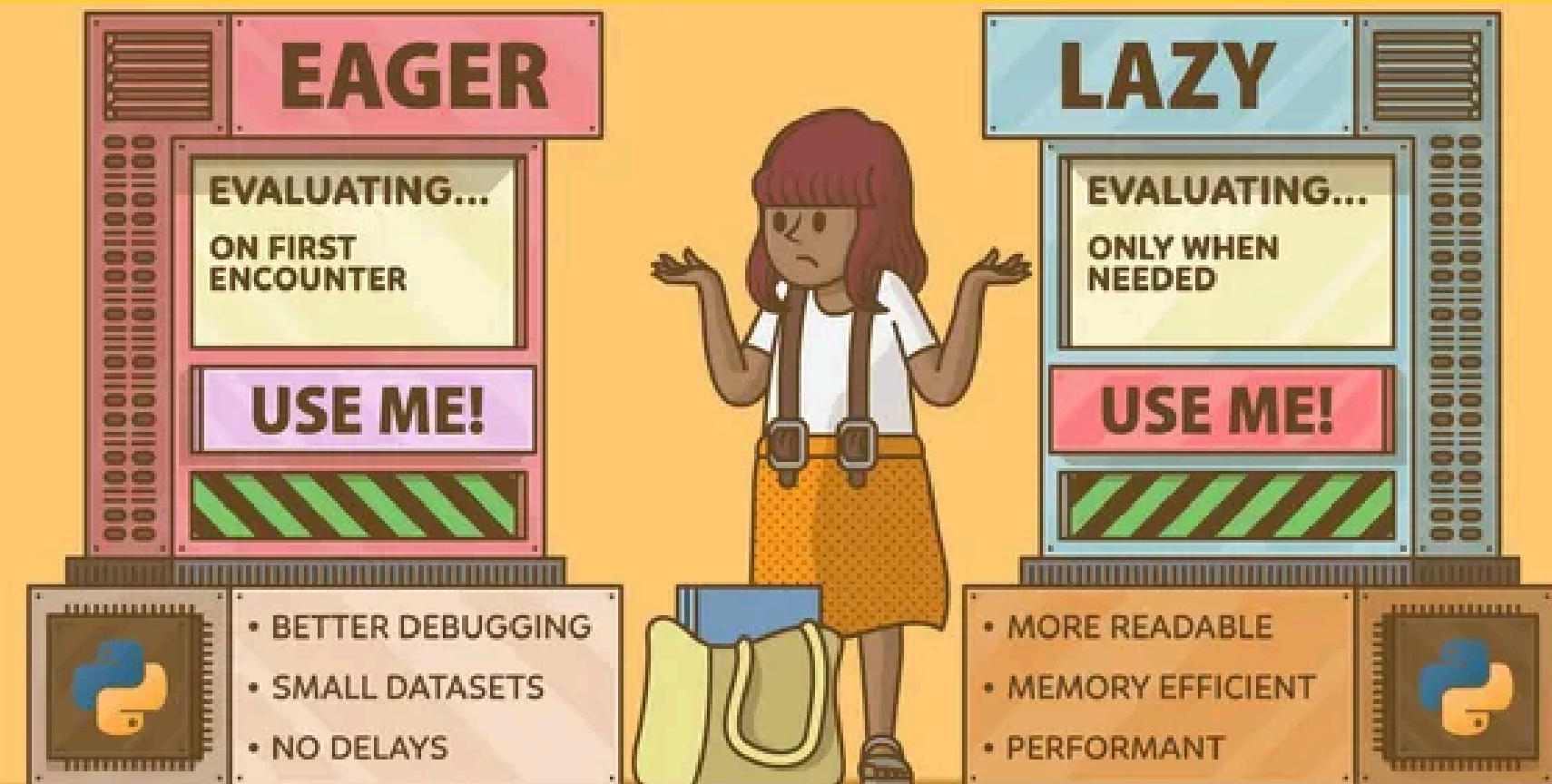
03

Function composition can be used to define new functions

FasterCapital

Lazy Evaluation

- Delay computation until the result is actually needed
- Ideal for huge payloads: ETL, REST integrations
- Saves memory, avoids thread congestion



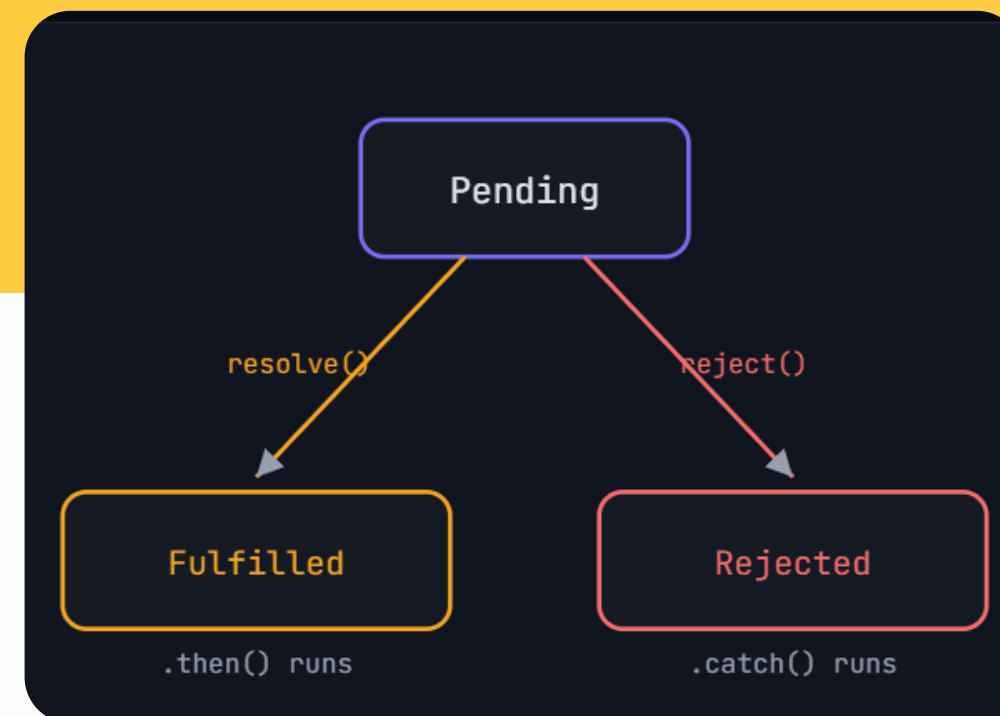
Non-Blocking / Asynchronous Programming



```
getUser(id, (user) => {  
  getOrders(user, (orders) => {  
    getInvoice(orders, (inv) => {  
      // finally, do the thing  
    })  
  })  
})  
the "pyramid of doom" - depth grows per step
```

Callbacks

- ✓ A function passed to run later, on completion
- ✗ Nesting many of them creates "callback hell"
- ✗ Error handling gets duplicated at every level



Promises

Style communicates feelings and moods. Color, form, and motion all play a role.

ASYNC / AWAIT

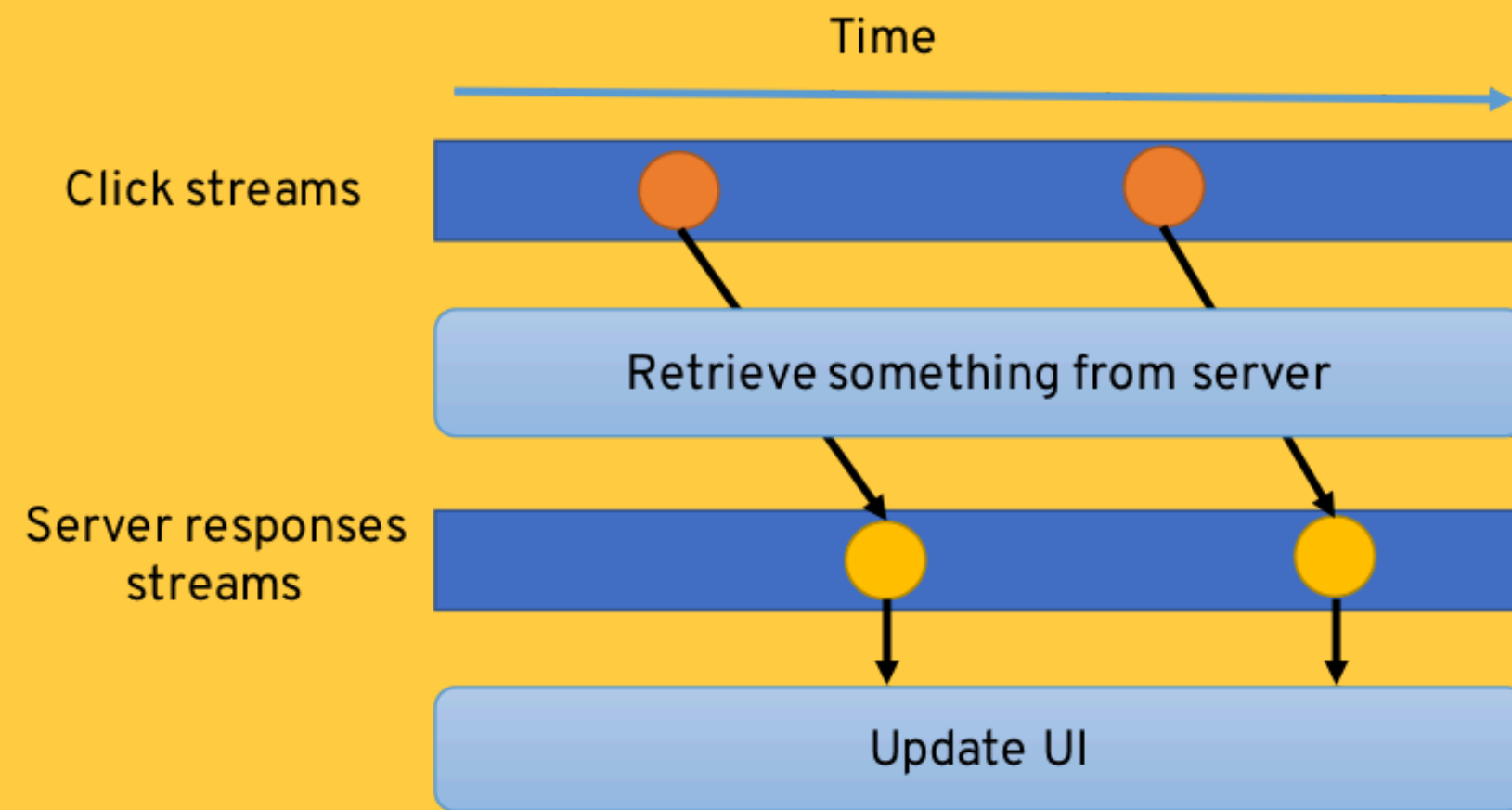
```
async function checkout(cart) {  
  try {  
    const u = await getUser(cart.userId);  
    const r = await charge(u, cart);  
    return confirm(r);  
  } catch (e) { log(e); }  
}
```

Async / Await

Good design balances creativity with structure. This harmony makes the work both practical and inspiring.

Reactive Programming

RxJS



Evolution of Promises & Callbacks



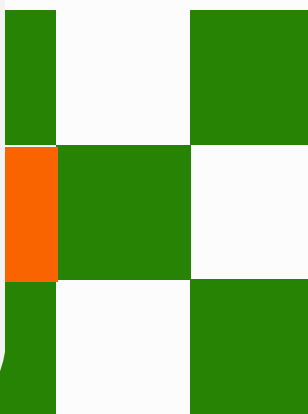
Async events modeled as streams over time



Subscribe & transform with pure operators



e.g. cursor tracking, keypresses, socket alerts



Enterprise Use Case

Very useful for high concurrency scenarios and input/output dependent operations.

Twitch

It is applied in the Chat system



Dixibrightwoods:

  Corny_And_Upset:

milkycowcoco:

   ennooaave: @dixibrightwoods

 oblivi_on:

Dixibrightwoods:

 aimsaiyan:

  Corny_And_Upset:

Pub-Sub and WebSocket



WebSocket

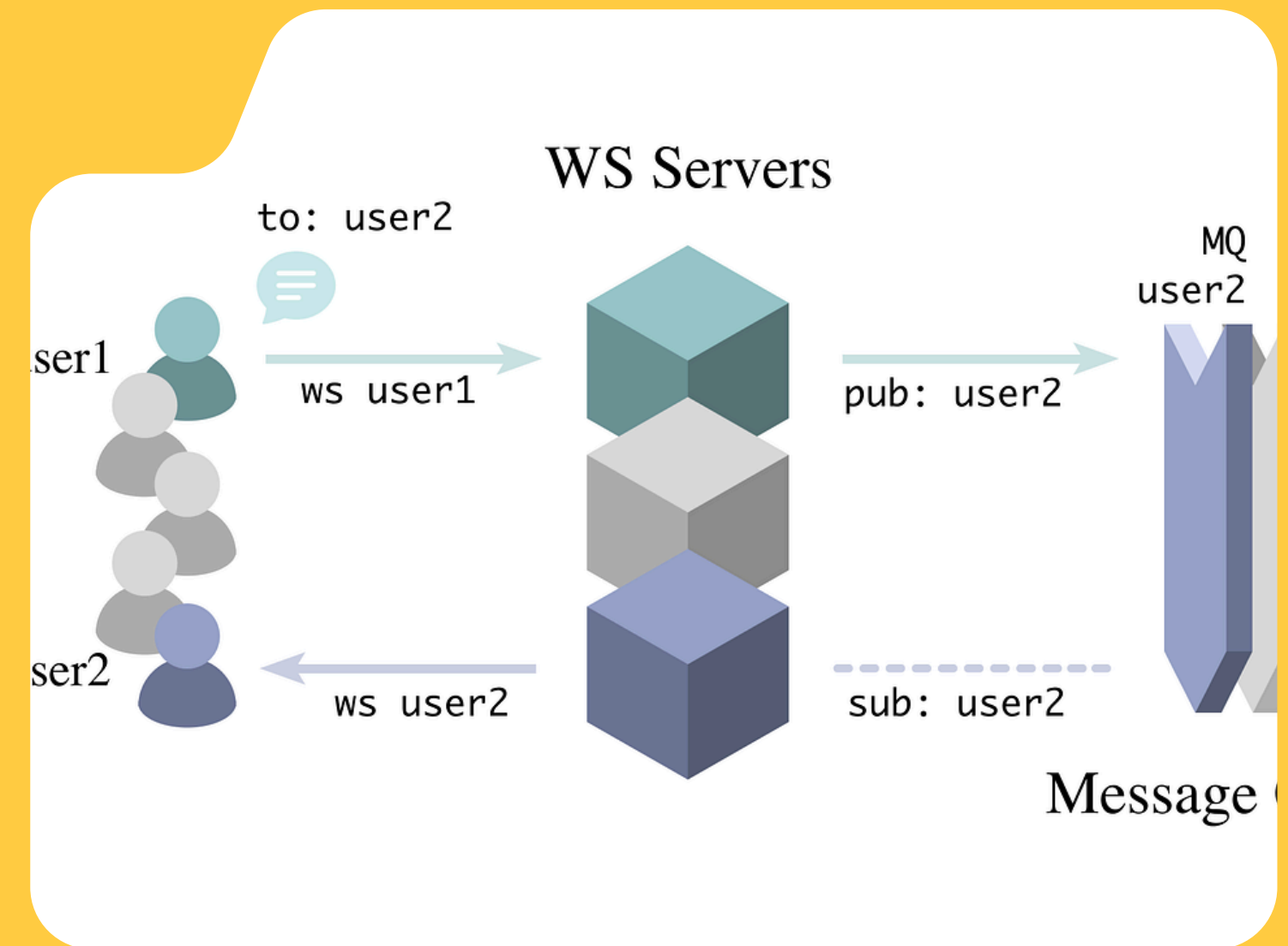
Network protocol that establishes a unique, persistent, bidirectional connection between the client and the server.

Publish

It is the user who sends the message

Subscribe

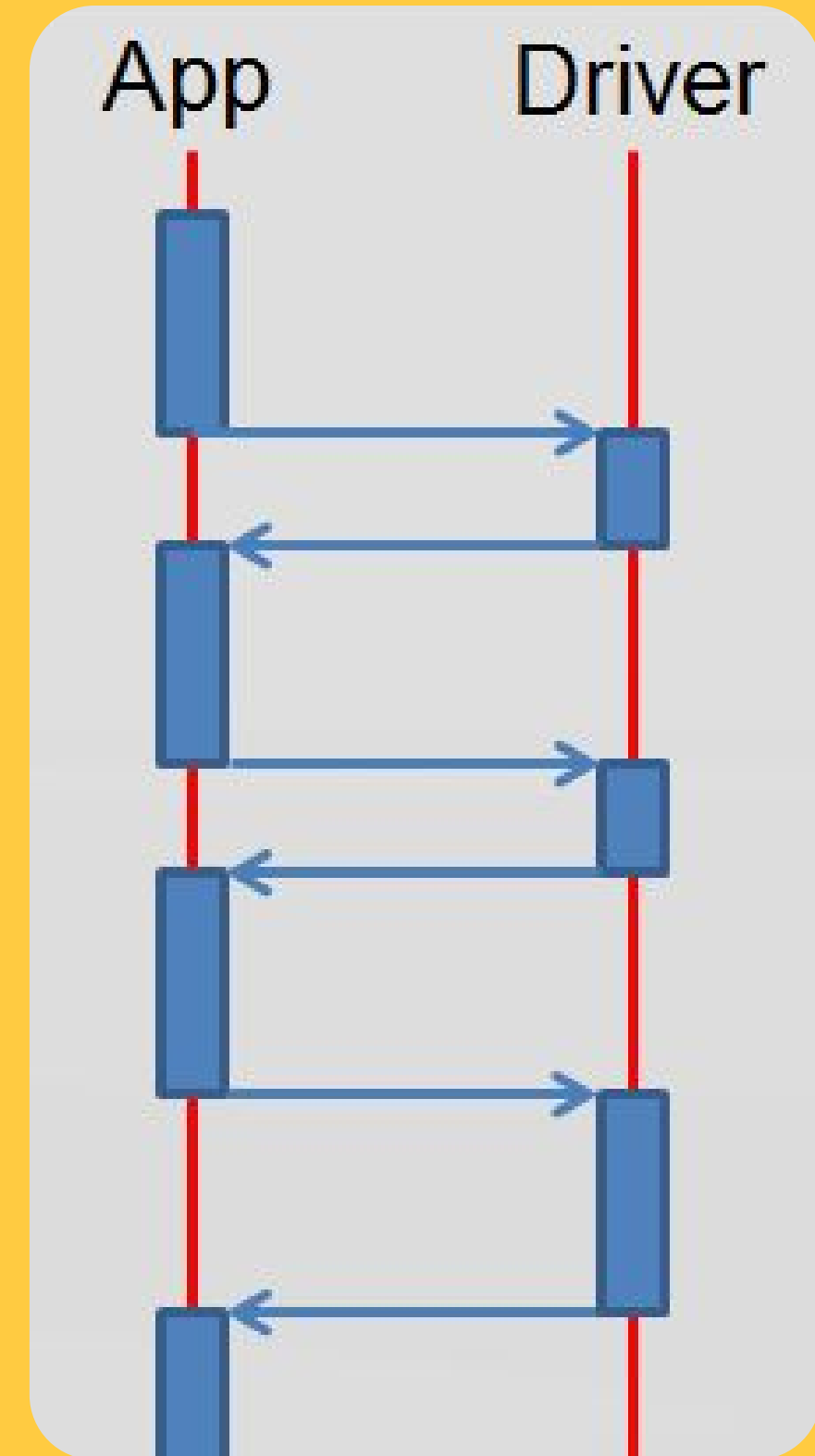
It is the user (or group of users) that listens to a channel





Where is it most useful and widely implemented?

Four fields where these
paradigms are widely
implemented.



High Concurrency & Real-Time

- Messaging apps, live dashboards, trading platforms
- Node.js, NestJS, Elixir
- Thousands of simultaneous I/O events, zero collapse

Data Processing & ETL

- Normalizing & analyzing large datasets
- MapReduce = map() + reduce()
- Immutability enables safe distributed processing



Modern Frontend Development

- Next.js & React rely on immutability to render
- New object created, never mutated in place
- Engine knows exactly what to update, asynchronously

Serverless Architectures

- Cloud functions (AWS Lambda) must be stateless
- Input in → output out, no global variables
- Same principle as a pure function



Applying to Biconoirs-Restaurant

In our project, we implemented non-blocking programming in the order section, at the time of generating the order.

Blocking

```
// Blocking version
const handleCheckout = async () => {
  let sum = 0;
  for (let i = 0; i < 1e10; i++) {
    sum += Math.sqrt(i);
  }

  const order = await cartStore.checkout();
  if (order) {
    toast.success('¡Pedido creado exitosamente!');
    router.push('/orders');
  } else if (cartStore.error) {
    toast.error(cartStore.error);
    cartStore.error = null;
  }
};
```

GRQV3

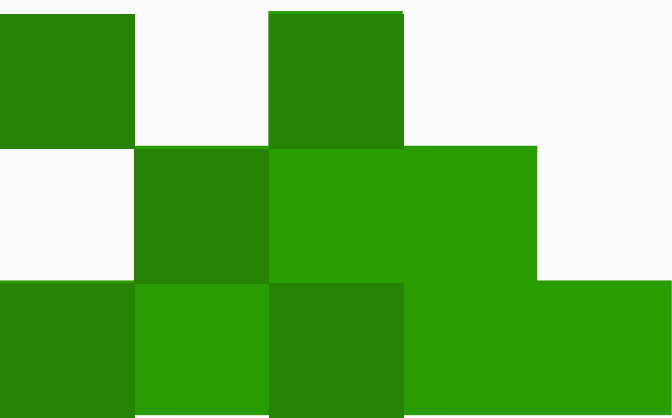
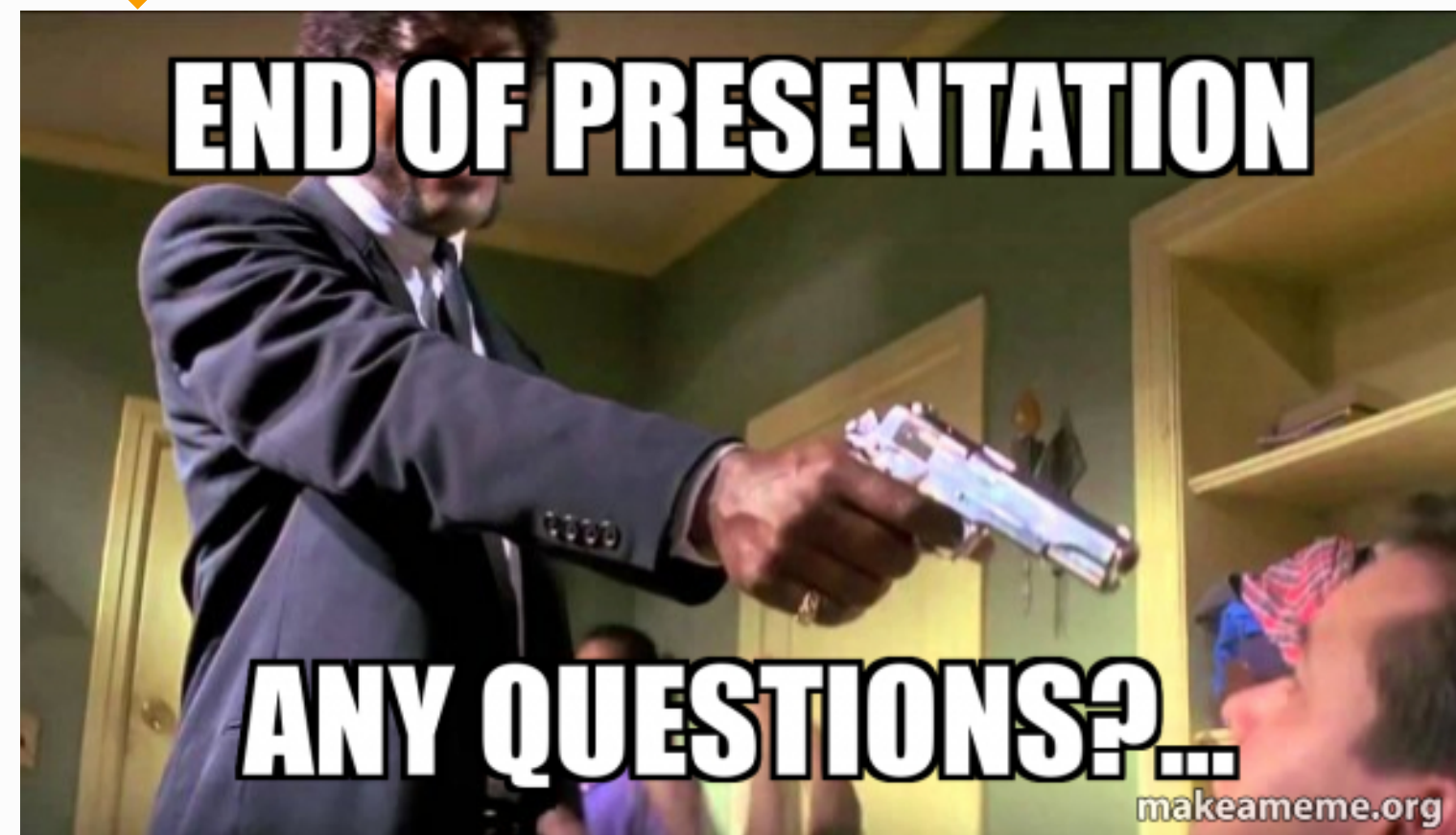
Non-Blocking

```
// Non-blocking version
const handleCheckout = async () => {

  const order = await cartStore.checkout();

  if (order) {

    toast.success('¡Pedido creado exitosamente!');
    router.push('/orders');
  } else if (cartStore.error) {
    router.push('/orders');
  } else if (cartStore.error) {
    toast.error(cartStore.error);
    cartStore.error = null;
  }
};
```



Thank you
so much

